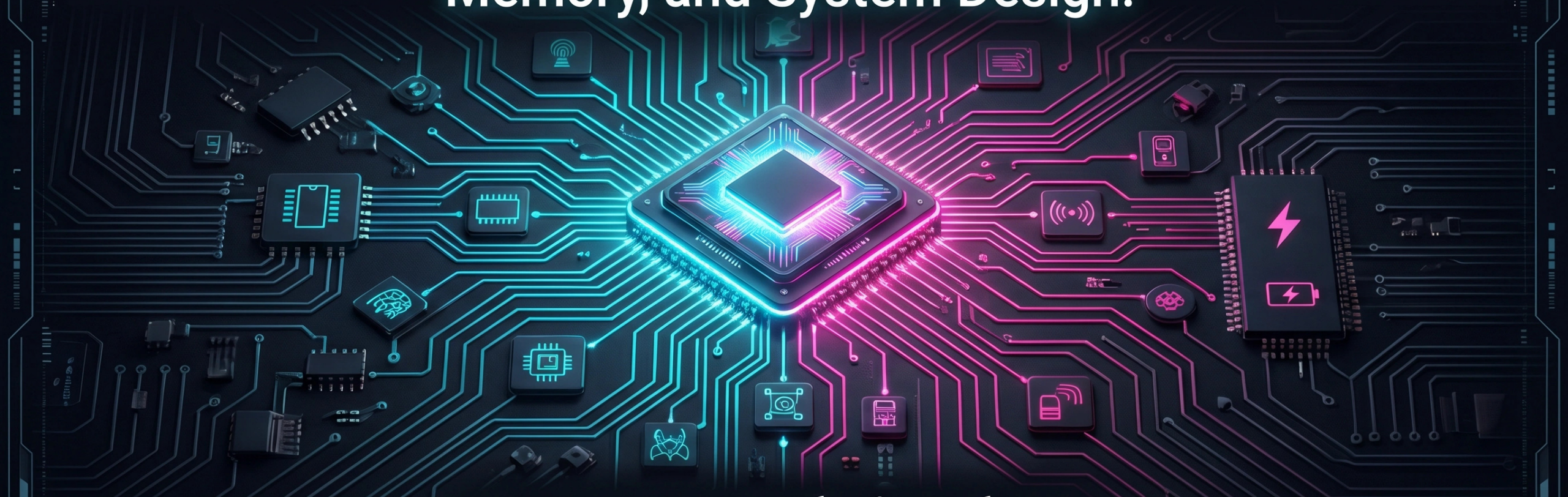
EMBEDDED OPERATING SYSTEMS ENGINEERING

From Bare Metal to RTOS — Scheduling, Interrupts, Memory, and System Design.



Murat Balci, PhD

What Makes Embedded OS Different from Desktop OS.

Comparing resource constraints, determinism, and operational models.

1. **Strict resource constraints:** limited RAM, flash, and CPU speed.
2. **Determinism is paramount:** guaranteed response times (hard vs. soft real-time).
3. **Autonomous operation:** no user keyboard, runs for years without reboot.
4. **Three execution models:** Bare Metal, RTOS, and Embedded Linux.
5. **Choice** depends on timing, complexity, memory, and peripherals.

Presenter: Engineering Lecture Series

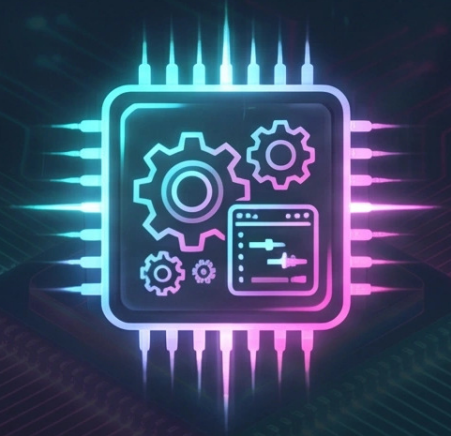
Bare Metal vs. RTOS vs. Embedded Linux

Comparison of Execution Models in Embedded Systems

Bare Metal (Superloop)

- Single main loop polling peripherals — simplest, lowest overhead, no scheduling.

RTOS (FreeRTOS, Zephyr)



- Lightweight kernel providing preemptive scheduling, IPC, and timing services (5-50 KB ROM).

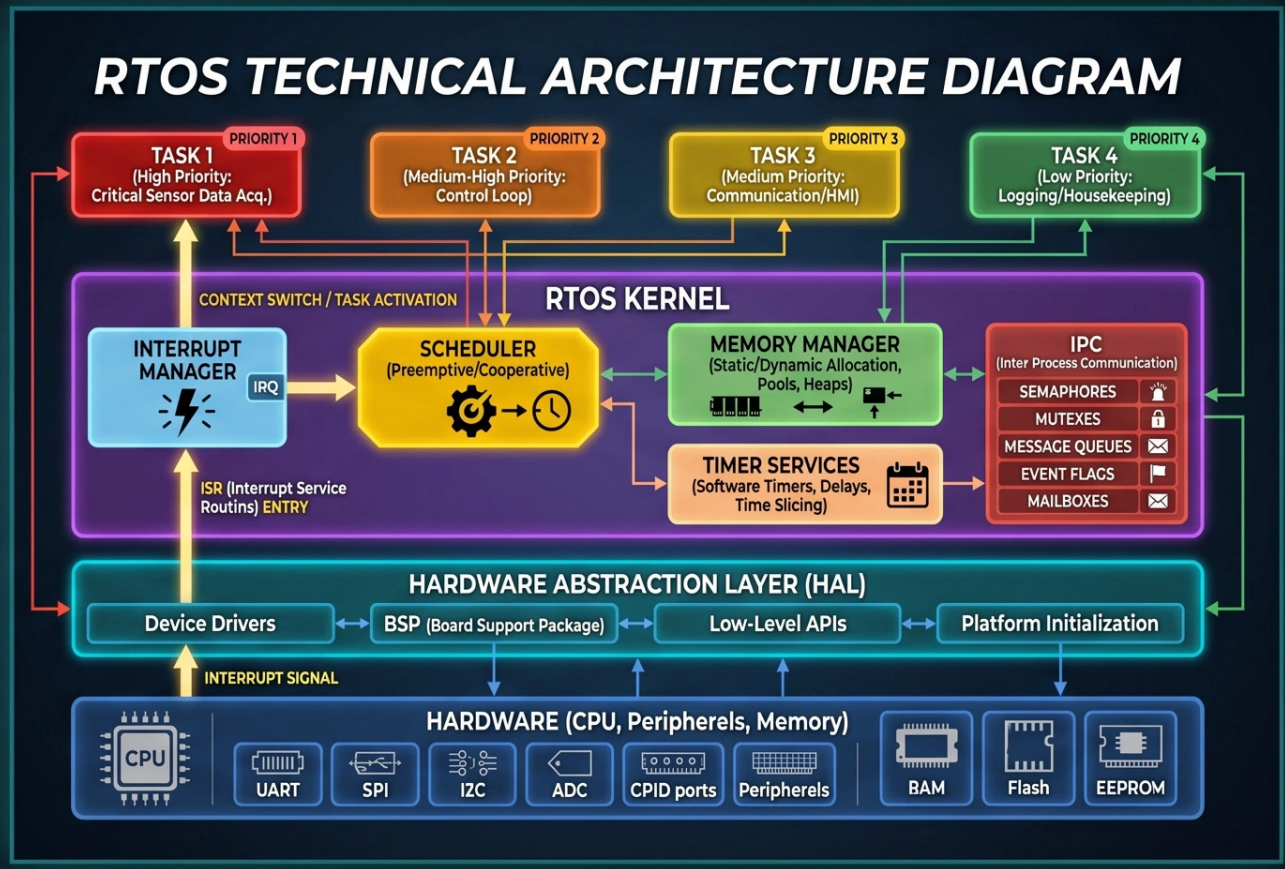
Embedded Linux

- Full POSIX OS with MMU, filesystem, networking, drivers — requires MB of RAM.

- **Key trade-off:** determinism and footprint (RTOS) vs. features and ecosystem (Linux).

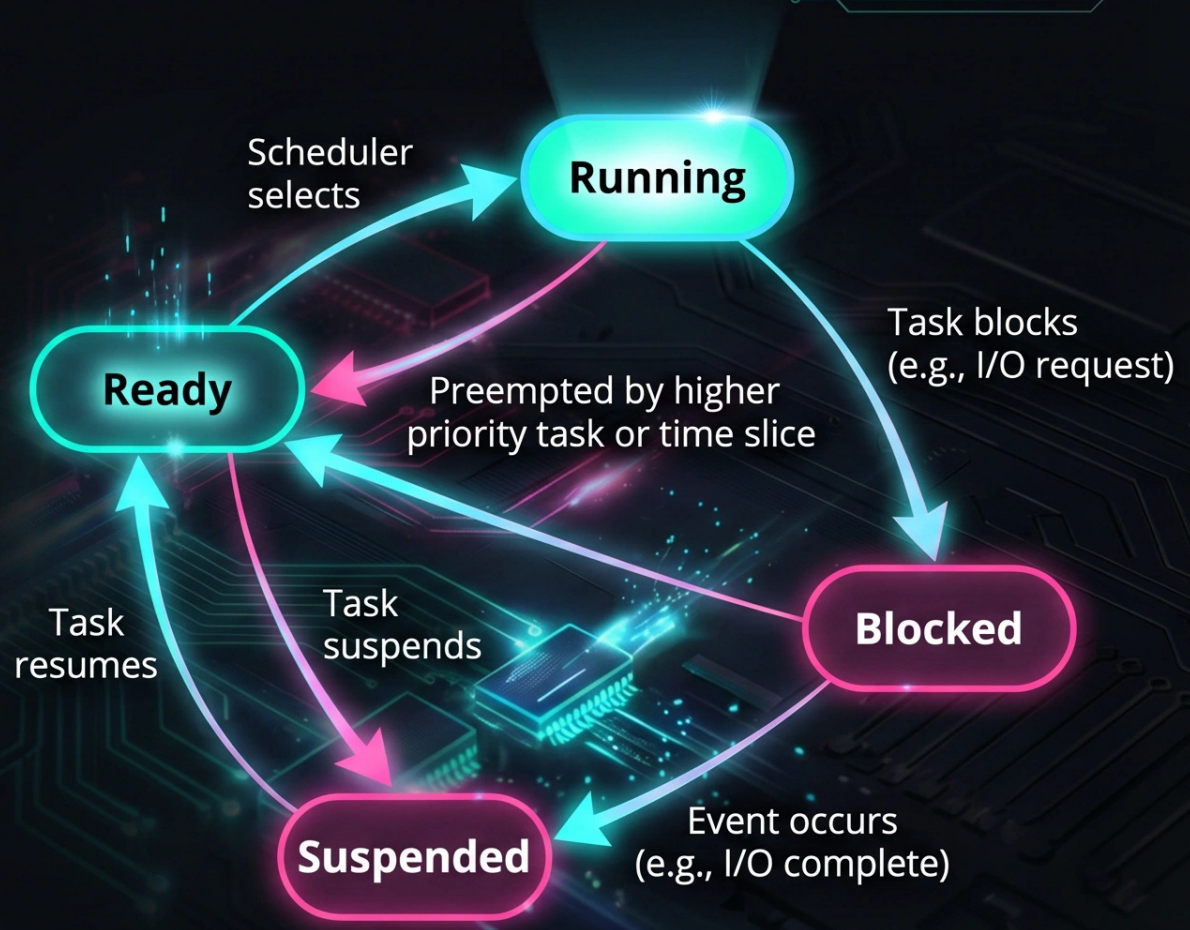
RTOS Architecture: Kernel Components

- The **RTOS kernel** is the core that manages tasks, timing, and resources.
- **Scheduler:** Decides which task runs next based on priority and state.
- **Interrupt Manager:** Routes hardware interrupts to ISRs and manages priorities.
- **Memory Manager:** Allocates memory from static pools or heap.
- **Timer Services:** Provides software timers, delays, and tick-based time slicing.
- **IPC Mechanisms:** Semaphores, mutexes, message queues, event flags.



Task Management and States

- A task is the fundamental unit of execution in an RTOS — each has its own stack, priority, and context.
- Task states: Ready, Running, Blocked, Suspended.
- The scheduler always runs the highest-priority ready task (preemptive scheduling).
- Tasks are created with a function pointer, stack size, and priority level.
- Context switching saves/restores CPU registers, stack pointer, and program counter.



Scheduling Algorithms: Choosing Who Runs When

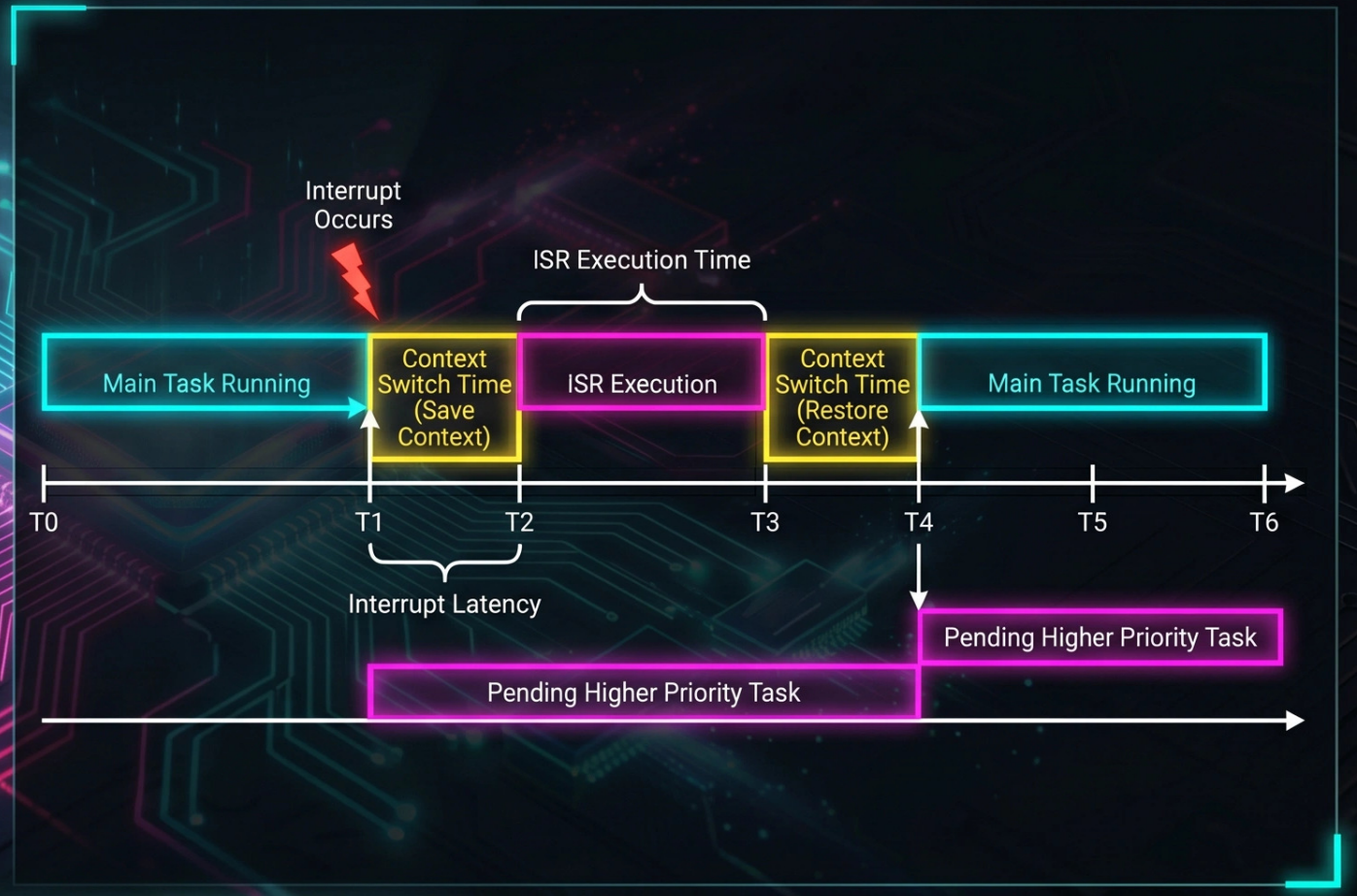
- 1. **Preemptive Priority-Based**: Highest-priority ready task always runs; lower-priority tasks are preempted immediately.
- 2. **Round-Robin (Time-Slicing):** Equal-priority tasks share CPU time in fixed time slices.
- 3. **Rate-Monotonic Scheduling (RMS):** Assigns priority based on task period (shorter period = higher priority).
- 4. **Earliest Deadline First (EDF):** Dynamic priority based on absolute deadline.
- 5. **Priority Inversion**: Solved by priority inheritance in mutexes.

Presenter: Engineering Lecture Series

Interrupt Handling: The Hardware-Software Bridge

Key Points

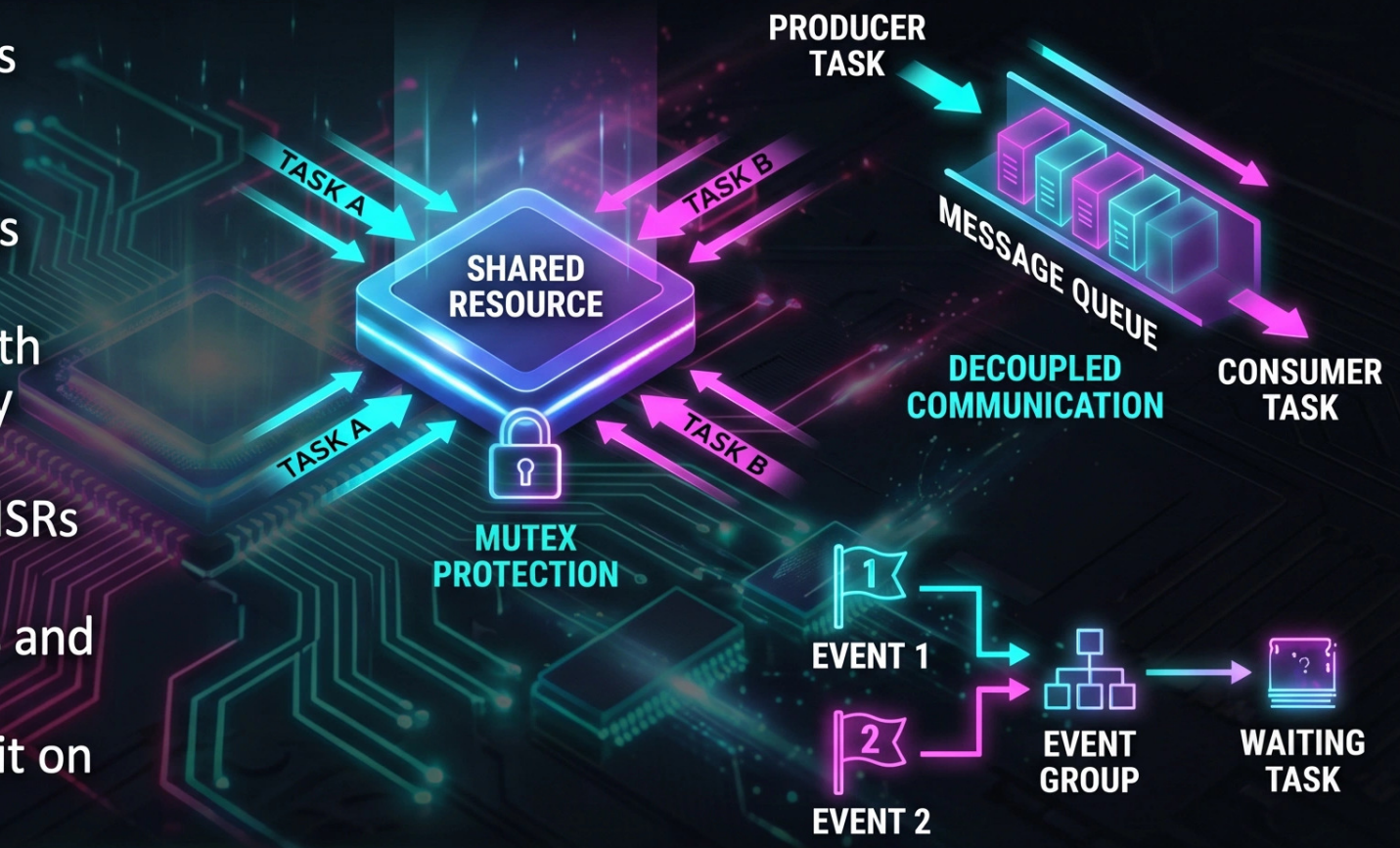
- An interrupt is a hardware signal that pauses the current task and transfers control to an ISR.
- Interrupt flow: Signal -> CPU saves context -> Vectors to ISR -> ISR executes -> Context restored -> Task resumes.
- Interrupt Latency = time from hardware signal to first ISR instruction.
- ISRs must be short and fast: defer complex work to tasks (top-half/bottom-half pattern).
- NVIC on ARM Cortex-M supports priority levels and nesting.



Critical Sections and Synchronization

Key Points:

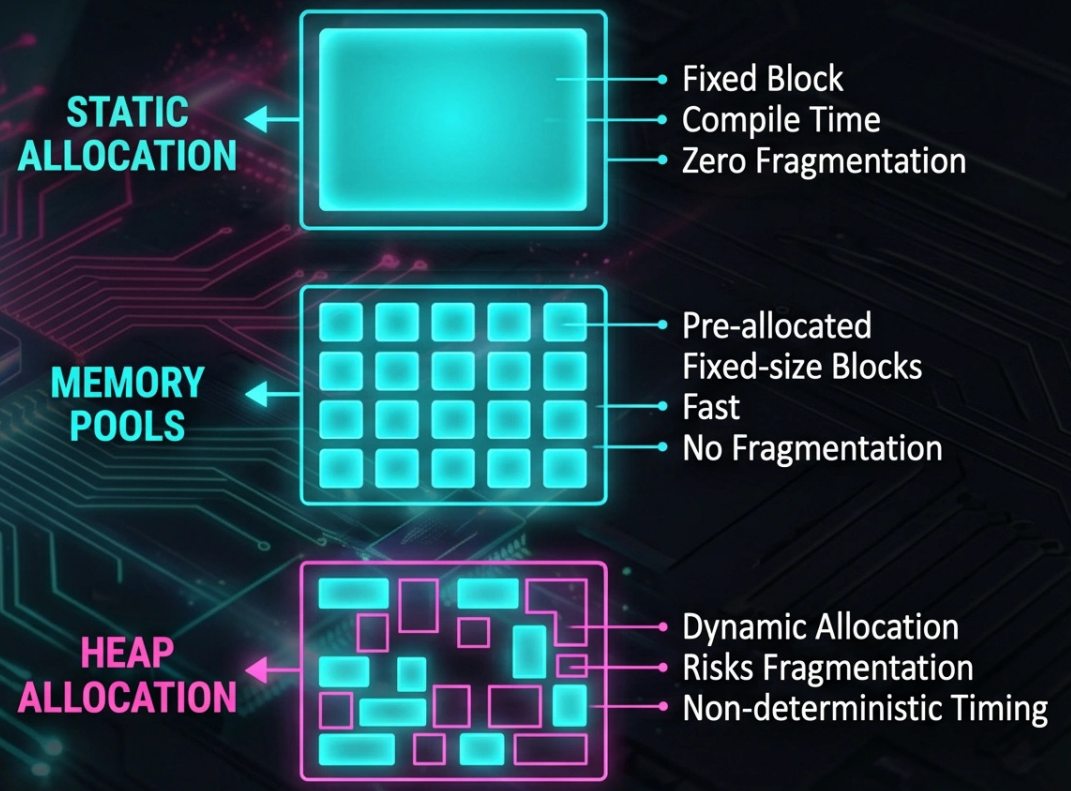
1. A **critical section** is code that accesses shared resources and must not be interrupted.
2. Simplest protection: disable interrupts (increases latency).
3. **Mutexes** provide mutual exclusion with priority inheritance to prevent priority inversion.
4. **Semaphores** signal between tasks or ISRs (binary or counting).
5. **Message Queues** decouple producers and consumers.
6. **Event Flags/Groups** allow tasks to wait on combinations of events.



Memory Management in Embedded Systems

Physical Addressing, Allocation Strategies, and Protection

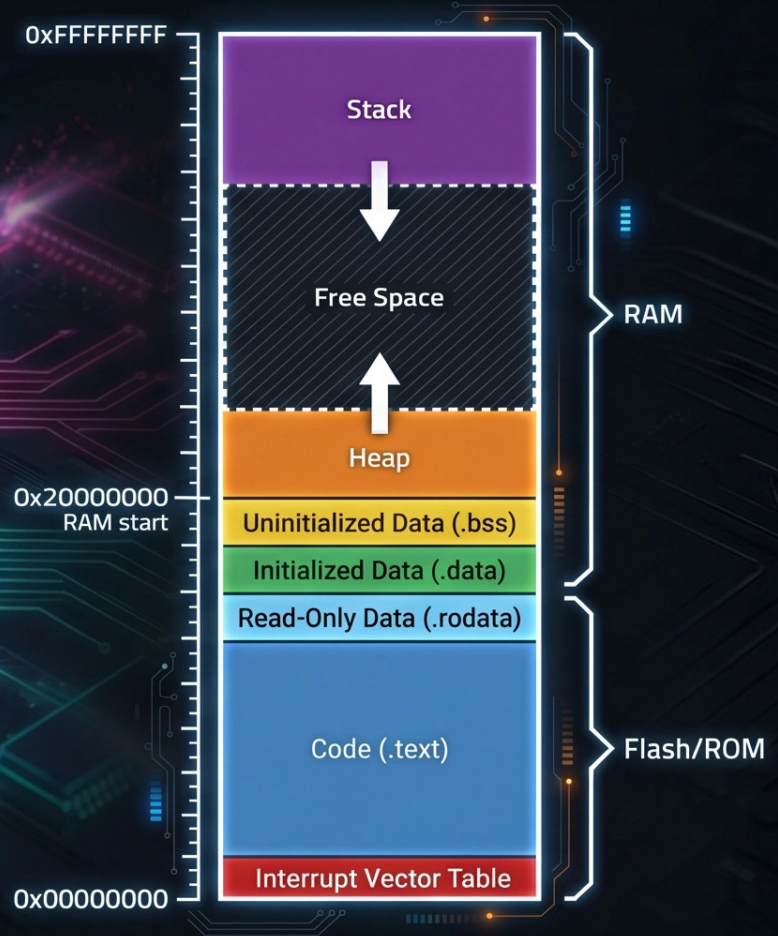
- 1. Embedded systems typically have no virtual memory — all addresses are physical.
- 2. Static Allocation: All memory determined at compile time — safest, no fragmentation.
- 3. Memory Pools: Pre-allocated fixed-size blocks — fast, no fragmentation.
- 4. Heap (malloc/free): Dynamic allocation — flexible but risks fragmentation and non-deterministic timing.
- 5. MPU (Memory Protection Unit): Hardware enforcing access permissions on memory regions.
- 6. Stack overflow detection: Guard patterns or MPU regions detect stack overflow.



Presenter: Engineering Lecture Series

Embedded Memory Layout: Flash and RAM

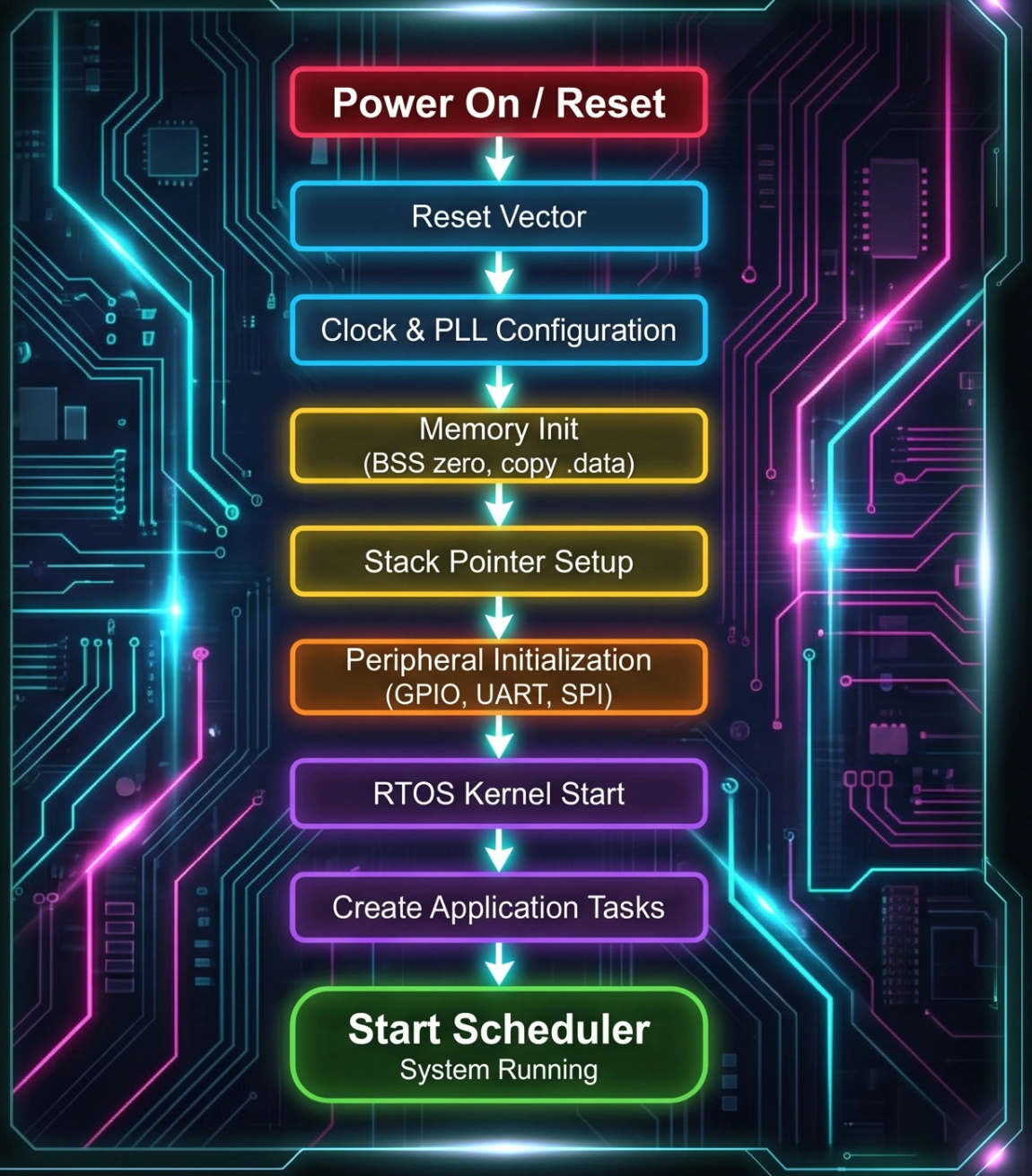
1. **Flash/ROM (0x00000000):** Contains interrupt vector table, executable code (.text), and read-only data (.rodata).
2. **RAM (0x20000000):** Contains initialized data (.data), uninitialized data (.bss), heap (grows up), and stack (grows down).
3. The linker script defines the memory map.
4. Startup code copies .data from flash to RAM and zeros .bss.
5. Stack and heap grow toward each other — collision means system failure.



Presenter: Engineering Lecture Series

Boot Sequence: From Power-On to Running Tasks

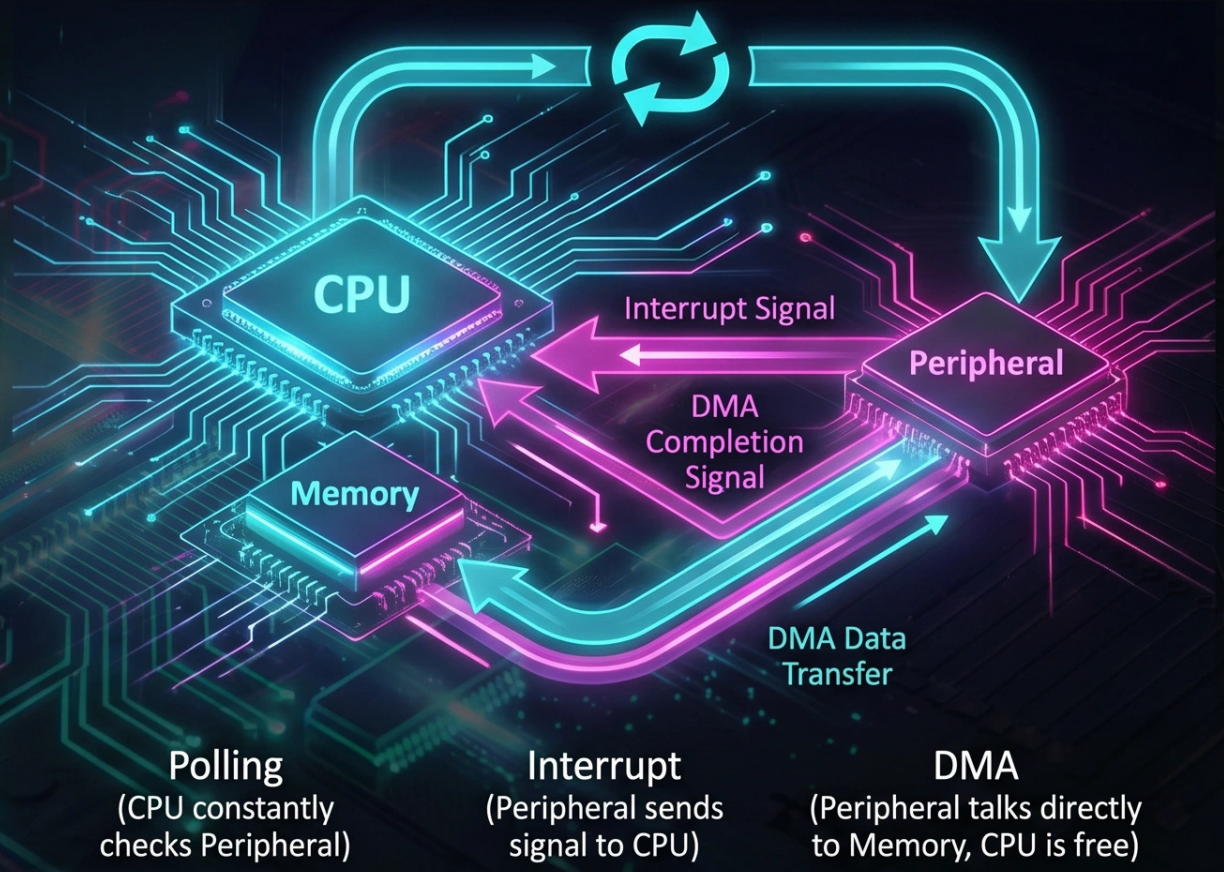
- **1. Power-On/Reset:** CPU loads initial stack pointer and reset vector.
- **2. Startup Code (Assembly):** Configures clock tree and PLL, initializes memory controller.
- **3. C Runtime Init:** Copies .data from flash to RAM, zeros .bss, sets up stack pointer.
- **4. Peripheral Init:** Configures GPIO, UART, SPI, I2C, timers.
- **5. RTOS Kernel Start:** Creates tasks, initializes kernel objects, starts scheduler.
- **6.** Once scheduler starts, RTOS takes control.



Device Drivers: Talking to Hardware.

- 1. Embedded drivers interact with peripherals through Memory-Mapped I/O (MMIO).
- 2. Driver responsibilities: initialization, configuration, data transfer, error handling.

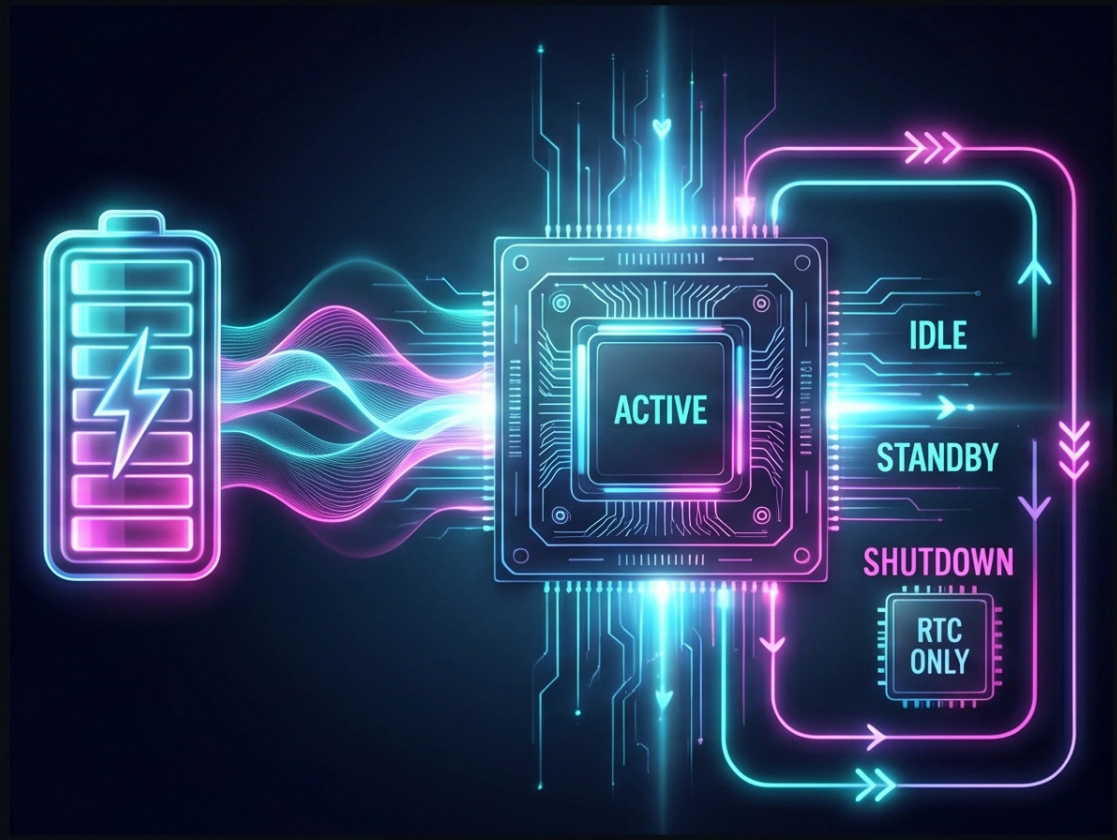
  3. Polling: CPU repeatedly checks status register — simple but wastes CPU cycles.
- 4. Interrupt-Driven: Hardware signals completion via interrupt — CPU is free to do other work.
- 5. DMA (Direct Memory Access): Dedicated hardware transfers data between peripherals and memory without CPU intervention.



Power Management: Extending Battery Life

Embedded devices often run on batteries — power management is a first-class design concern.

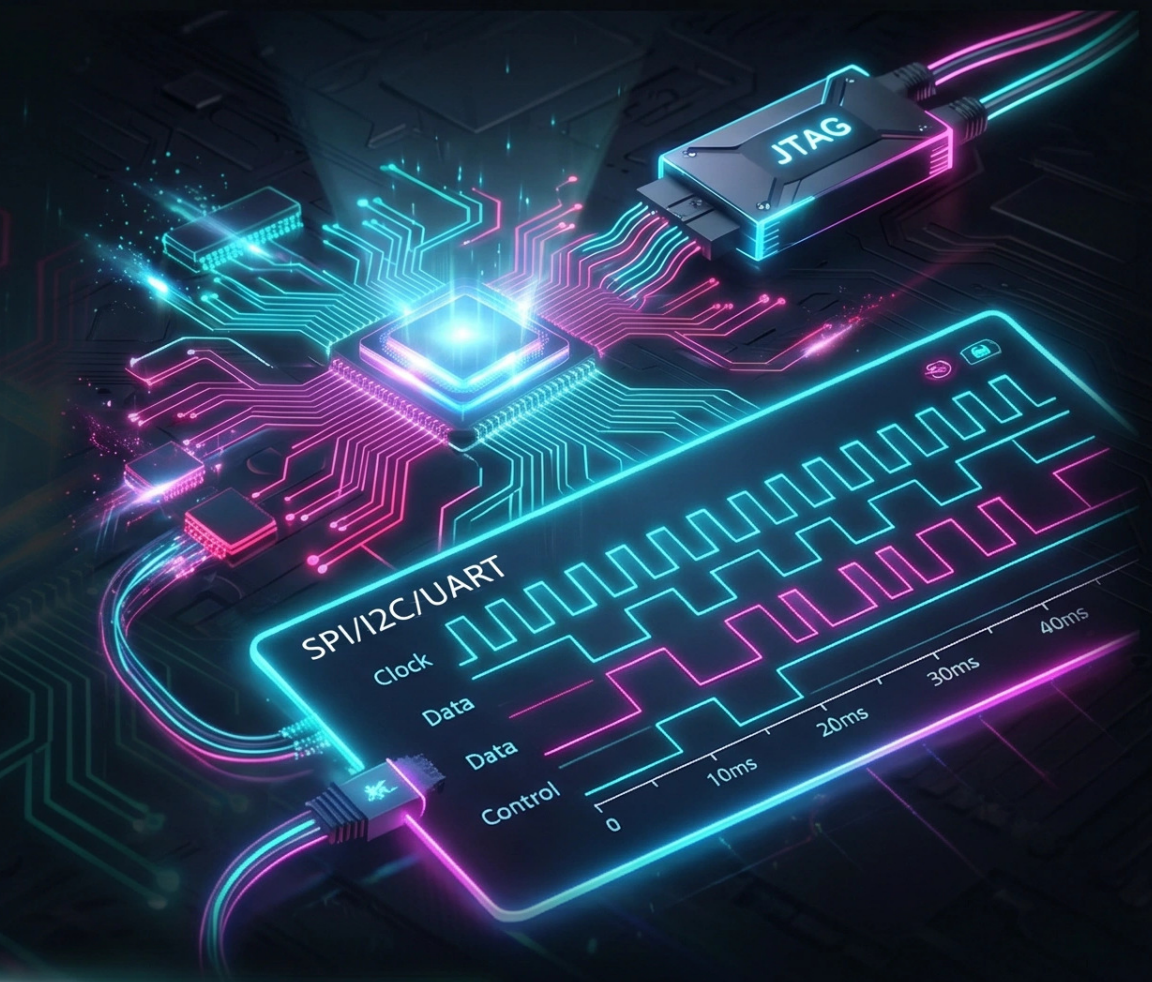
- **Sleep Modes:** Idle (CPU halted), Standby (clocks stopped, RAM retained), Shutdown (only RTC powered).
- **Clock Gating:** Disabling clocks to unused peripherals reduces dynamic power.
- **Dynamic Voltage and Frequency Scaling (DVFS):** Reducing clock speed/voltage during low demand.
- **Wake-Up Sources:** Interrupts, RTC alarms, and peripheral events.
- **RTOS tickless idle mode** stops the system tick during idle periods to maximize sleep.



Debugging and Testing Embedded Systems

Key Points:

1. **JTAG/SWD:** Hardware debug interfaces for breakpoints, single-stepping, and memory inspection.
2. **Printf Debugging:** UART serial output for logging — simple but intrusive.
3. **Logic Analyzers and Oscilloscopes:** Capture bus signals (SPI, I2C, UART) and timing behavior.
4. **RTOS-Aware Debugging:** Tools visualize task scheduling, interrupts, and IPC in real time.
5. **Hardware-in-the-Loop (HIL):** Simulates the physical environment while testing the real embedded system.



Presenter: Engineering Lecture Series

Common RTOS Platforms and Ecosystem

1. **FreeRTOS:** Most popular open-source RTOS — minimal kernel, huge community, AWS IoT integration.
2. **Zephyr:** Linux Foundation-backed RTOS with built-in networking, Bluetooth, and device tree support.
3. **VxWorks:** Commercial RTOS for safety-critical systems (aerospace, medical, automotive).
4. **ThreadX (Azure RTOS):** Ultra-small footprint, safety-certified, now open-source under Eclipse Foundation.
5. **Embedded Linux (Yocto/Buildroot):** Full OS for complex embedded applications.

Summary: Key Principles for Embedded OS Engineers

1. **Determinism over throughput:** Guarantee worst-case response times.
2. **Resource awareness:** Every byte matters — prefer static allocation.
3. **Interrupt discipline:** Keep ISRs short, defer work to tasks.
4. **Concurrency safety:** Protect shared resources with mutexes, use queues.
5. **Power consciousness:** Design for sleep, minimize active time.
6. Understanding these fundamentals enables building reliable systems.